

MindSync AI

Complete System Documentation

AI Mental Wellness Companion for IT Professionals

Viva & Presentation Preparation MasterBook

Generated: August 15, 2026 at 08:45 AM

Version 1.0.0 | Backend: FastAPI + scikit-learn + SHAP + Groq LLM | Frontend: Flutter + Riverpod + Firebase

Total Backend Files: 25 | Total Frontend Files: 130+ | Test Files: 7 | Total Lines of Code: 8000+

Table of Contents

PART I: SYSTEM OVERVIEW

1. Project Introduction & Problem Statement
2. System Architecture (Three-Tier Design)
3. Technology Stack & Dependencies
4. Complete Project Directory Structure

PART II: BACKEND - FILE BY FILE CODE DOCUMENTATION

5. Application Bootstrap (main.py)
6. Configuration & Settings (core/config.py)
7. Firebase Service Integration (core/firebase.py)
8. JWT Authentication & Security (core/security.py)
9. Exception Handlers (core/exceptions.py)
10. Structured Logging (core/logging.py)
11. Prompt Library (core/prompt_library.py)
12. Rule-Based Notification Engine (core/notification_engine.py)
13. Middleware: Request Logging (middleware/logging_middleware.py)
14. Middleware: Rate Limiting (middleware/rate_limit_middleware.py)
15. Middleware: Security Headers (middleware/security_headers_middleware.py)
16. ML Training Pipeline (ml/train.py)
17. ML Burnout Prediction Pipeline (ml/pipeline.py)
18. Hybrid Recommendation Engine (ml/recommendation_engine.py)
19. AI Orchestrator Service (services/ai_orchestrator.py)
20. Model Manager & MLOps (services/model_manager.py)
21. Cache Service (services/cache_service.py)
22. Background Scheduler (services/scheduler.py)
23. AI Security Manager (utils/ai_security.py)
24. Standardized Response Utility (utils/responses.py)
25. API Endpoint: Health Check (api/endpoints/health.py)
26. API Endpoint: Burnout Prediction (api/endpoints/predict.py)
27. API Endpoint: Recommendations (api/endpoints/recommendations.py)
28. API Endpoint: Analytics & Reports (api/endpoints/analytics.py)

29. API Endpoint: Notifications (api/endpoints/notifications.py)

PART III: FRONTEND - FLUTTER APP DOCUMENTATION

30. Flutter App Entry Point (main.dart)

31. App Configuration & Environment (core/config)

32. Routing & Navigation (core/routing)

33. Theme System (core/theme)

34. Network Layer & API Client (core/network)

35. Feature Modules Overview (Clean Architecture)

36. Authentication Feature

37. Dashboard Feature

38. Mood Tracking Feature

39. AI Chat Coach Feature

40. Recommendations Feature

41. Reports & Analytics Feature

42. Notifications Feature

43. Profile & Settings Features

PART IV: DATABASE & SECURITY

44. Firestore Database Schema & Security Rules

45. Firebase Authentication Flow

PART V: DEPLOYMENT & DEVOPS

46. Docker Containerization

47. Render Cloud Deployment

PART VI: TESTING

48. Backend Test Suite (7 Test Files)

PART VII: AI/ML DEEP DIVE

49. Complete ML Pipeline Walkthrough

50. SHAP Explainability System

51. Groq LLM Integration

PART VIII: USER GUIDE - ALL FEATURES

52. Complete Feature-by-Feature User Guide

PART IX: VIVA PREPARATION

53. 50 Potential Viva Questions with Model Answers

PART I: SYSTEM OVERVIEW

1. Project Introduction & Problem Statement

Project Name: MindSync AI - AI Mental Wellness Companion for IT Professionals

Problem Statement: Software engineers and IT professionals face alarmingly high rates of occupational burnout, stress, and mental fatigue due to prolonged screen time, tight deadlines, continuous learning pressure, and sedentary work habits. According to research, 83% of developers experience burnout. Traditional wellness apps are generic and do not address the unique challenges of the IT industry.

Proposed Solution: MindSync AI is a production-grade, AI-driven mental wellness companion specifically designed for IT professionals. It combines classical Machine Learning (Random Forest) for burnout risk prediction, Generative AI (Groq LLM) for creative personalized recommendations, and a deterministic Rules Engine for instant threshold-based alerts. The system uses a Flutter mobile app as the frontend and a FastAPI Python backend with Firebase cloud services.

Key Innovation: Three-tier hybrid AI architecture combining ML prediction, LLM-generated content, and rule-based deterministic logic -- ensuring the system works even when AI services are unavailable.

1.1 Project Objectives

- * Predict burnout risk using a trained Random Forest model on 9 behavioural features
- * Provide explainable AI predictions using SHAP (SHapley Additive exPlanations)
- * Generate personalised wellness recommendations via Groq LLM (LLaMA 3.3-70B)
- * Enable daily mood tracking with comprehensive wellness metric logging
- * Deliver AI-powered chat coaching using developer-friendly language
- * Compile visual analytics dashboards with trend analysis and reports
- * Implement rule-based smart notifications for hydration, sleep, and stress
- * Support PDF/CSV report export for wellness data
- * Ensure production-grade security (JWT auth, rate limiting, prompt injection defence)
- * Deploy via Docker containers to Render cloud platform

2. System Architecture (Three-Tier Design)

MindSync AI follows a **client-server architecture** with three distinct AI tiers:

Layer	Technology	Purpose
Frontend (Client)	Flutter + Dart + Riverpod	Cross-platform mobile UI, state management, local Hive storage
Backend (Server)	FastAPI + Python	REST API, ML inference, LLM orchestration, business logic
Database (Cloud)	Firebase (Firestore + Auth)	User authentication, cloud data persistence, real-time sync
AI Tier 1 - ML	scikit-learn Random Forest	Burnout risk classification (Low/Medium/High)
AI Tier 2 - GenAI	Groq API (LLaMA 3.3-70B)	Creative wellness recommendations & summaries
AI Tier 3 - Rules	Python if-else logic	Deterministic threshold-based notifications

Key Insight: The three-tier AI design ensures graceful degradation: if the LLM API is down, ML predictions and rule-based recommendations still work. If the ML model fails to load, heuristic fallback rules maintain basic functionality.

2.1 Data Flow Architecture

- * **Step 1:** User logs wellness metrics (mood, sleep, stress, steps, water, exercise) via Flutter app
- * **Step 2:** Data is stored in Firebase Firestore and sent to FastAPI backend via Dio HTTP client
- * **Step 3:** Backend validates input via Pydantic schemas and authenticates via JWT (Firebase ID Token)
- * **Step 4:** ML Pipeline (BurnoutPipeline) scales features with RobustScaler and runs Random Forest inference
- * **Step 5:** SHAP TreeExplainer identifies top contributing factors for the prediction
- * **Step 6:** HybridRecommendationEngine generates rules-based + ML-based + LLM-generated recommendations
- * **Step 7:** RuleBasedNotificationEngine evaluates thresholds and generates contextual alerts
- * **Step 8:** Response sent back to Flutter app, which renders dashboards, charts, and recommendations

3. Technology Stack & Dependencies

3.1 Backend Technologies

Library	Version	Role
FastAPI	>=0.110.0	Async web framework for REST API endpoints
uvicorn	>=0.28.0	ASGI server running the FastAPI application
Pydantic	>=2.6.0	Data validation & schema enforcement for request/response models
scikit-learn	>=1.4.0	ML models: RandomForest, LogisticRegression, GridSearchCV, RobustScaler, metrics
pandas	>=2.2.0	DataFrame operations for feature engineering and model input
numpy	>=1.26.0	Numerical computing: synthetic data generation, array operations
shap	>=0.45.0	Explainable AI: TreeExplainer for per-prediction feature contributions
jobjlib	>=1.3.0	Model serialization: save/load .pkl model artifacts
requests	>=2.31.0	Synchronous HTTP for Groq API (recommendation engine)
httpx	>=0.27.0	Async HTTP for Groq API (AI orchestrator)
firebase-admin	>=6.5.0	Firebase Admin SDK for auth token verification & Firestore
PyJWT	>=2.8.0	JWT token handling for authentication fallback
loguru	>=0.7.2	Structured logging with colored output and file rotation
reportlab	>=4.0.0	PDF report generation with styled tables and formatted content
python-dotenv	>=1.0.1	Load .env environment variables for configuration

3.2 Frontend Technologies

Package	Version	Role
Flutter SDK	>=3.4.0	Cross-platform mobile app framework (Dart language)
flutter_riverpod	^2.5.1	State management & dependency injection
go_router	^14.2.0	Declarative routing with auth-based redirects
dio	^5.4.3	HTTP client for backend API communication
firebase_core/auth/firestore	^3.3/^5.1/^5.2	Firebase SDK for auth, database, cloud storage
hive/hive_flutter	^2.2.3	Fast local key-value NoSQL database for offline caching
fl_chart	^0.68.0	Data visualization: line/bar/pie charts for analytics
google_fonts	^6.2.1	Custom typography (Inter font family)

Package	Version	Role
lottie	^3.1.2	Lottie JSON animations for UI micro-interactions
shimmer	^3.0.0	Loading skeleton placeholders
geolocator	^12.0.0	Location services for weather integration
sensors_plus/pedometer	^5.0.1/^4.0.1	Hardware sensor access for step counting
flutter_local_notifications	^17.1.2	Local push notification scheduling

4. Complete Project Directory Structure

```

MindSync AI/<br/>+-- backend/                                # FastAPI Python Backend<br/>|   +-- app/<br/>|   |   +-- main.py                                # Application bootstrap & startup<br/>|   |   |   +-- __init__.py                        # Package initializer<br/>|   |   |   |   +-- api/endpoints/                # REST API route handlers<br/>|   |   |   |   |   +-- health.py                 # Health check & diagnostics<br/>|   |   |   |   |   |   +-- predict.py            # /predict/burnout endpoint<br/>|   |   |   |   |   |   |   +-- recommendations.py # Wellness recommendations<br/>|   |   |   |   |   |   |   |   +-- analytics.py   # Analytics, trends, reports<br/>|   |   |   |   |   |   |   |   |   +-- notifications.py # Notification CRUD & evaluation<br/>|   |   |   |   |   |   |   |   |   |   +-- core/   # Core infrastructure<br/>|   |   |   |   |   |   |   |   |   |   |   +-- config.py # Environment settings<br/>|   |   |   |   |   |   |   |   |   |   |   |   +-- init<br/>|   |   |   |   |   |   |   |   |   |   |   |   |   +-- security.py # JWT auth middleware<br/>|   |   |   |   |   |   |   |   |   |   |   |   |   |   +-- exceptions.py # Global exception handlers<br/>|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   +-- logging.py # Loguru structured logging<br/>|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   +-- notification_engine.py # Rule-based alerts<br/>|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   +-- middleware/ # HTTP middleware stack<br/>|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   +-- logging_middleware.py # Request/response logging<br/>|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   +-- rate_limit_middleware.py # IP-based rate limiting<br/>|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   +-- security_headers_middleware.py # OWASP headers<br/>|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   +-- ml/ # Machine Learning<br/>|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   +-- train.py # Model training script<br/>|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   +-- pipeline.py # Inference pipeline<br/>|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   +-- recommendation_engine.py # Hybrid recs engine<br/>|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   +-- models/burnout_model.pkl # Trained model (~4.8MB)<br/>|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   +-- services/ # Business services<br/>|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   +-- ai_orchestrator.py # LLM API management<br/>|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   +-- model_manager.py # Model loading & health<br/>|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   +-- cache_service.py # TTL in-memory cache<br/>|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   +-- scheduler.py # Background async tasks<br/>|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   +-- ai_security.py # Prompt injection defence<br/>|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   +-- responses.py # JSON response wrapper<br/>|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   +-- tests/ # Pytest test suite (7 files)<br/>|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   +-- requirements.txt # Python dependencies<br/>|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   +-- Dockerfile # Container image spec<br/>|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   +-- docker-compose.yml # Local dev orchestration<br/>|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   +-- frontend/ # Flutter Mobile App<br/>|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   +-- lib/<br/>|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   +-- main.dart # App entry point<br/>|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   +-- core/ # Core infra (config, routing, theme, network)<br/>|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   +-- features/ # 10 feature modules (Clean Architecture)<br/>|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   +-- authentication/ # Login, register, splash, onboarding<br/>|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   +-- dashboard/ # Main dashboard with 11 widget cards<br/>|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   +-- mood/ # Mood logging, journal, statistics<br/>|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   +-- chat/ # AI wellness coach chat<br/>|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   +-- burnout/ # Burnout prediction display<br/>|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   +-- recommendations/ # AI-generated recommendations<br/>|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   +-- reports/ # Analytics charts & PDF export<br/>|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   +-- notifications/ # Notification center & settings<br/>|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   +-- profile/ # User profile management<br/>|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   +-- settings/ # App, privacy, security settings<br/>|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   +-- shared/ # Shared widgets & providers<br/>|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   +-- pubspec.yaml # Flutter dependencies<br/>|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   +-- firestore.rules # Firestore security rules<br/>|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   +-- render.yaml # Render cloud deployment config<br/>|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   +-- docs/ # Documentation files<br/>

```

PART II: BACKEND - FILE BY FILE CODE DOCUMENTATION

5. Application Bootstrap - main.py (68 lines)

Purpose: This is the entry point of the entire FastAPI backend application. It initialises all services, registers middleware, and mounts API routers.

Line-by-Line Explanation:

- * **Lines 1-14 (Imports):** Imports uvicorn (ASGI server), FastAPI framework, CORS middleware, all core modules (config, logging, exceptions, firebase), all three middleware classes, all five API endpoint routers, the ML pipeline, scheduler, and model manager.
- * **Line 17: setup_logging()** - Configures Loguru as the global logger, intercepting all uvicorn and FastAPI logs.
- * **Lines 19-23: FastAPI app creation** - Creates the FastAPI application instance with project name, version, and description from settings.
- * **Line 26: setup_exception_handlers(app)** - Registers three global exception handlers: HTTPException, RequestValidationError, and generic Exception.
- * **Lines 29-38: Middleware stack registration** - Adds 4 middleware layers in order: CORSMiddleware (cross-origin requests), RequestLoggingMiddleware (audit trail), RateLimitingMiddleware (60 req/min/IP), SecurityHeadersMiddleware (OWASP headers).
- * **Lines 41-57: @app.on_event('startup')** - The startup lifecycle hook runs these in sequence: (1) FirebaseService.initialize() - connects Firebase Admin SDK, (2) ModelManager().initialize() - loads ML model with SHA-256 verification, (3) BackgroundScheduler().start() - launches async cleanup/refresh loops, (4) BurnoutPipeline().initialize() - pre-loads the trained Random Forest model into memory.
- * **Lines 60-64: Router registration** - Mounts 5 API routers: health (no prefix), predict, recommendations, analytics, notifications (all under /api prefix).
- * **Lines 66-67: Direct execution** - Runs uvicorn server on 0.0.0.0:8000 with hot-reload enabled.

Key Insight: The middleware execution order is REVERSE of registration order. Requests flow: SecurityHeaders -> RateLimit -> Logging -> CORS -> Handler. Responses flow back in reverse.

6. Configuration & Settings - core/config.py (35 lines)

Purpose: Centralises all application configuration using environment variables with sensible defaults. Uses python-dotenv to load .env files.

- * **Line 5: load_dotenv()** - Loads variables from .env file into os.environ at import time.
- * **Lines 7-34: Settings class** - Defines all configuration constants:
 - * - PROJECT_NAME: Application display name (default: 'MindSync AI Wellness Companion')
 - * - VERSION: Semantic version string (default: '1.0.0')
 - * - API_STR: URL prefix for all API routes ('/api')
 - * - ENV: Runtime environment ('development', 'testing', or 'production')
 - * - ALLOWED_ORIGINS: CORS whitelist, comma-separated (default: '*' allows all)
 - * - FIREBASE_CREDENTIALS_PATH: Local path to Firebase service account JSON

- * - GROQ_API_KEY: API key for Groq LLM inference
- * - GEMINI_API_KEY: Legacy Google Gemini key (fallback)
- * - RATE_LIMIT_PER_MINUTE: Max API requests per IP per 60 seconds (default: 60)
- * - API_KEY_PROTECTION: Whether to enforce API key header validation
- * - SECURE_HEADERS_ENABLED: Toggle OWASP security headers (default: true)
- * **Line 34: settings = Settings()** - Creates a singleton instance imported by all modules.

7. Firebase Service Integration - core/firebase.py (52 lines)

Purpose: Manages Firebase Admin SDK initialization with 3-method fallback chain for maximum deployment flexibility.

- * **Lines 7-9: Class attributes** - `_initialized` flag prevents double-init; `db` holds Firestore client reference.
- * **Lines 12-14: Guard clause** - If already initialised, returns immediately (idempotent).
- * **Lines 16-27: Method 1: JSON Environment Variable** - Reads `FIREBASE_CREDENTIALS_JSON` env var (used for Render cloud). Parses JSON string into dict, creates Certificate credential, initializes app.
- * **Lines 30-38: Method 2: File Path** - Reads `FIREBASE_CREDENTIALS_PATH` env var pointing to a local JSON file (used for local development).
- * **Lines 41-46: Method 3: Default** - Falls back to initializing with just `projectId` (limited functionality, no Firestore write access).
- * **Lines 48-51: Firestore client** - Attempts to create Firestore client; sets `db=None` if it fails (app continues without database).

Key Insight: The 3-method fallback chain is critical for portability: JSON env vars for cloud (Render), file paths for local dev, and bare project ID as last resort.

8. JWT Authentication & Security - core/security.py (58 lines)

Purpose: Implements Firebase JWT token verification with a manual JWT decode fallback for cloud environments.

- * **Line 9: security = HTTPBearer()** - Creates FastAPI security scheme requiring 'Authorization: Bearer <token>' header.
- * **Lines 11-24: _decode_jwt_payload_fallback(token)** - Manual JWT payload extraction: splits token by '.', base64-decodes the middle segment (payload), extracts `user_id/sub/uid` fields. This is used when Firebase Admin cannot verify tokens on Render (missing service account).
- * **Lines 26-56: get_current_user_id(credentials)** - The main auth function used as a FastAPI `Depends()` dependency:
- * - **Mock token check (line 30):** Tokens starting with 'mock_' or 'dev_' return 'usr_mock_123' (developer sandbox)
- * - **Primary path (lines 36-41):** `firebase_auth.verify_id_token(token)` - Full cryptographic verification
- * - **Fallback path (lines 43-46):** If Firebase verification fails, tries manual JWT decode to extract UID
- * - **Error path (lines 48-51):** Raises HTTP 401 Unauthorized with descriptive error message

9. Exception Handlers - core/exceptions.py (41 lines)

Purpose: Registers three global exception handlers to ensure ALL errors return standardised JSON responses (never raw HTML stack traces in production).

- * **HTTPException handler:** Catches FastAPI HTTP errors (400, 401, 404, etc.) and wraps them in standardized_response format with success=false.
- * **RequestValidationError handler:** Catches Pydantic validation failures (422). Extracts each error's location and message, returns them in data.details array.
- * **Generic Exception handler:** Catches ALL unhandled exceptions. Logs full stack trace via logger.exception() and returns HTTP 500 with generic message (hides internal details from users).

10. Structured Logging - core/logging.py (39 lines)

Purpose: Replaces Python's default logging with Loguru for structured, coloured, production-grade logging.

- * **InterceptHandler class:** A custom logging.Handler that intercepts all standard library log records and redirects them to Loguru's logger.
- * **setup_logging():** (1) Replaces root handler with InterceptHandler, (2) Intercepts uvicorn.error, uvicorn.access, and fastapi loggers, (3) Configures Loguru with green timestamps, coloured log levels, cyan file/function/line info.

11. Prompt Library - core/prompt_library.py (34 lines)

Purpose: Centralises all LLM prompt templates in a single versioned location.

Template	Purpose	Variables
daily_summary	Generate daily wellness report	mood_score, stress_level, sleep_hours, water_intake, exercise_minutes
weekly_report	Analyse 7-day trend data	logs (serialised history)
chat_coaching	AI wellness coach chat responses	message (user input)
motivational_quote	Developer-themed motivational quotes	None

12. Rule-Based Notification Engine - core/notification_engine.py (95 lines)

Purpose: Evaluates user wellness metrics against predefined threshold rules to generate contextual push notifications.

Rule	Condition	Title	Priority
1	stress >= 8 AND sleep < 6h	Nervous System Reset	Critical
2	burnout_risk == 'High'	Unplug & Recover	Critical
3	sleep < 6 hours	Sleep Recovery Reminder	High
4	water < 6 glasses	Hydration Reminder	Medium
5	exercise < 20 min	Active Movement Stretch	Medium
6	mood <= 3	Mindful Check-in	High
Default	No rules triggered	Daily Wellness Check	Low

13. Middleware: Request Logging - logging_middleware.py (30 lines)

Purpose: Logs every incoming HTTP request and outgoing response with method, path, client IP, status code, and processing time in milliseconds. Creates a complete audit trail.

14. Middleware: Rate Limiting - rate_limit_middleware.py (37 lines)

Purpose: Implements IP-based sliding window rate limiting. Stores request timestamps per client IP and blocks requests exceeding RATE_LIMIT_PER_MINUTE (default: 60). Uses a 60-second sliding window. Health endpoint (/api/health) and testing environment are exempt.

15. Middleware: Security Headers - security_headers_middleware.py (16 lines)

Purpose: Adds OWASP-recommended security headers to every HTTP response when SECURE_HEADERS_ENABLED is true.

Header	Value	Protection
X-Frame-Options	DENY	Prevents clickjacking by blocking iframe embedding
X-Content-Type-Options	nosniff	Prevents MIME-type sniffing attacks
X-XSS-Protection	1; mode=block	Enables browser XSS filter
Strict-Transport-Security	max-age=31536000; includeSubDomains; preload	Forces HTTPS for 1 year
Content-Security-Policy	default-src 'self'; frame-ancestors 'none'	Restricts resource loading sources
Referrer-Policy	no-referrer	Prevents sending referrer information

16. ML Training Pipeline - ml/train.py (158 lines)

Purpose: Generates synthetic wellness data, trains multiple ML models, tunes hyperparameters, and exports the best model as a .pkl artifact.

16.1 generate_synthetic_dataset() Function

- * Generates 10,000 synthetic wellness records using NumPy random distributions
- * Each record has 9 features representing IT professional daily metrics
- * A domain-expert-weighted formula computes a continuous risk score (0-100)
- * Gaussian noise (sigma=4.0) is added to prevent perfectly deterministic class boundaries
- * Records are labelled: <40 = Low (0), 40-70 = Medium (1), 70+ = High (2)

16.2 train_and_evaluate() Function

- * **Step 1:** Calls generate_synthetic_dataset() to create DataFrame with 10K records
- * **Step 2:** Separates features (X) from labels (y); drops risk_score column
- * **Step 3:** train_test_split(test_size=0.2, stratify=y) creates 80/20 split maintaining class distribution
- * **Step 4:** RobustScaler() fitted on training data; transforms both train and test sets
- * **Step 5:** Defines 4 candidate models: LogisticRegression, DecisionTree, RandomForest, GradientBoosting
- * **Step 6:** Selects Random Forest for tuning based on domain guidelines
- * **Step 7:** GridSearchCV tests 18 combinations (n_estimators: [50,100,150], max_depth: [6,10,None], min_samples_split: [2,5]) with 3-fold CV using f1_macro scoring
- * **Step 8:** Evaluates tuned model on test set; prints accuracy, F1-score, and classification report
- * **Step 9:** Initializes SHAP TreeExplainer for global feature importance
- * **Step 10:** Serializes model+scaler+features+metadata to burnout_model.pkl via joblib.dump()

17. Burnout Prediction Pipeline - ml/pipeline.py (169 lines)

Purpose: Singleton class that loads the trained model and performs real-time burnout risk inference with SHAP explanations.

17.1 BurnoutPipeline Class

- * **Singleton Pattern (__new__):** Ensures only one instance exists application-wide, preventing redundant model loading
- * **initialize():** Loads burnout_model.pkl via joblib.load(); extracts model, scaler, feature_names, metadata; initialises SHAP TreeExplainer
- * **predict(input_data) - 12-Step Process:**
 - * 1. Checks if initialised; calls initialize() if needed (lazy loading)
 - * 2. Maps 9 features to correct trained order
 - * 3. Fills missing features with sensible defaults (sleep=7, steps=5000, etc.)
 - * 4. Converts to pandas DataFrame with correct column names
 - * 5. Scales features using trained RobustScaler.transform()

- * 6. `model.predict()` returns predicted class index (0/1/2)
- * 7. `model.predict_proba()` returns probability distribution across 3 classes
- * 8. Extracts confidence = probability of predicted class
- * 9. Computes continuous risk score: $P(\text{Medium}) \times 50 + P(\text{High}) \times 100$ (range 0-100)
- * 10. `SHAP explainer.shap_values()` computes per-feature contributions
- * 11. Top 3 positive SHAP contributors mapped to human-readable labels
- * 12. Risk-appropriate wellness recommendations generated

18. Hybrid Recommendation Engine - ml/recommendation_engine.py (300 lines)

Purpose: Combines three recommendation sources (Rules + ML + LLM) into a unified, prioritised recommendation list.

- * **Source 1 - Rules Engine:** 4 threshold rules: low sleep (<6h), high stress (>7), low water (<6 glasses), low exercise (<20min)
- * **Source 2 - ML-Driven:** If burnout risk is High/Medium, adds 'Mandatory Disconnection' recommendation
- * **Source 3 - Groq LLM:** Sends user metrics to LLaMA 3.3-70B with structured prompt; requests 2 creative JSON recommendations
- * **Fallback:** If LLM fails, returns hardcoded 'IT Workday Screen Break' recommendation
- * **Report Generators:** `generate_daily_summary()`, `generate_weekly_report()`, `generate_monthly_insights()` produce structured wellness reports

19. AI Orchestrator Service - services/ai_orchestrator.py (91 lines)

Purpose: Manages all Groq LLM API interactions with caching, retry logic, and graceful fallback.

- * **generate_summary():** Checks CacheService first; builds prompt from PromptLibrary templates; calls Groq API; caches result (daily=1h TTL, weekly=12h TTL)
- * **_execute_groq_request_with_retry():** 3-retry async HTTP call with exponential backoff (1s, 2s, 4s). Uses httpx.AsyncClient. Returns fallback string on total failure.
- * **Mock detection:** If API key is placeholder/mock, returns static response immediately without making API calls.

20. Model Manager - services/model_manager.py (78 lines)

Purpose: Singleton service for secure ML model loading with SHA-256 integrity verification and heuristic fallback.

- * **initialize():** Computes SHA-256 checksum of .pkl file before loading; loads model, scaler, features, metadata; stores checksum in metadata
- * **_load_fallback_heuristics():** If model file missing/corrupt, sets model=None and provides rule-based backup with estimated 80% accuracy
- * **get_model_health():** Returns diagnostic dict: initialized, version, type (primary_ml/heuristic), accuracy, checksum

21. Cache Service - services/cache_service.py (30 lines)

Purpose: In-memory TTL (Time-To-Live) cache implemented as class-level dictionary mapping keys to (value, expiry_timestamp) tuples.

22. Background Scheduler - services/scheduler.py (54 lines)

Purpose: Singleton async task scheduler running two background loops: nightly cache cleanup (24h) and recommendation refresh (12h). Uses asyncio.create_task() for non-blocking execution.

23. AI Security Manager - utils/ai_security.py (41 lines)

Purpose: Defends against prompt injection attacks with 3 layers: HTML tag stripping, 7 regex blacklist patterns, and automatic medical disclaimer injection.

24. Standardized Response - utils/responses.py (20 lines)

Purpose: Ensures all API responses follow a consistent JSON structure: {success, message, data, timestamp, requestId}. The requestId is a UUID v4 for request tracing.

25. API: Health Check - health.py (48 lines)

Endpoint	Method	Auth	Purpose
/health	GET	None	Basic health check returning status, timestamp, version
/health/details	GET	None	Detailed diagnostics: API, database, AI service, ML model, background worker status

26. API: Burnout Prediction - predict.py (66 lines)

Endpoint	Method	Auth	Purpose
/api/predict/burnout	POST	JWT	Accepts 9 wellness metrics, returns burnoutRisk, confidence, riskScore, importantFactors, recommendations

Input Schema (PredictionRequest): sleepHours (0-24), workingHours (0-24), moodScore (1-10), stressLevel (1-10), energyLevel (1-10), waterIntake (0-30 glasses or ml), dailySteps (0+), exerciseMinutes (0-1440), consecutiveWorkingDays (0-365). All validated by Pydantic with ge/le constraints.

27. API: Recommendations - recommendations.py (138 lines)

Endpoint	Method	Auth	Purpose
/api/recommendations/generate	POST	JWT	Generates hybrid recommendations (rules + ML + LLM)
/api/recommendations/daily-summary	POST	JWT	Returns daily wellness score and achievements
/api/recommendations/weekly-report	GET	JWT	Returns weekly trend analysis and top recommendations
/api/recommendations/monthly-insights	GET	JWT	Returns monthly wellness insights
/api/recommendations/feedback	POST	JWT	Logs user feedback on recommendations

28. API: Analytics & Reports - analytics.py (649 lines)

Purpose: The largest backend file. Computes wellness analytics, trend data, historical comparisons, and generates PDF/CSV reports.

Endpoint	Method	Purpose
/api/analytics/summary	POST	Computes overall wellness score, averages, goal completion rate from client-provided mood logs
/api/analytics/trends	POST	Returns time-filtered trend data with daily wellness scores, mood, stress, sleep, hydration, burnout risk
/api/analytics/historical-stats	POST	Compares current week vs previous week, current month vs previous month wellness scores
/api/analytics/report/generate	POST	Generates comprehensive wellness report JSON with AI insights, mood/stress/sleep/activity/hydration analysis
/api/analytics/report/export	POST	Exports report as PDF (using ReportLab) or CSV with formatted tables and styled content

Wellness Score Formula: 30% Mood + 20% Stress (inverse) + 20% Sleep (cap 8h) + 15% Hydration (cap 8 glasses) + 15% Exercise (cap 30min) = 0-100 score

29. API: Notifications - notifications.py (220 lines)

Endpoint	Method	Purpose
GET /api/notifications	GET	Returns all notifications for user (seeds defaults on first access)
POST /api/notifications/{id}/read	POST	Marks a specific notification as read
DELETE /api/notifications/{id}	DELETE	Deletes a specific notification
POST /api/notifications/preferences	POST	Updates notification preferences (push, email, quiet hours, etc.)
GET /api/notifications/preferences	GET	Returns current notification preferences
POST /api/notifications/sync	POST	Syncs notification history from client to server
POST /api/notifications/trigger-evaluation	POST	Triggers rule engine evaluation on current metrics

PART III: FRONTEND - FLUTTER APP DOCUMENTATION

30. Flutter App Entry Point - main.dart (107 lines)

- * **CrashGuard (lines 18-44):** Three layers of crash prevention: FlutterError.onError catches framework errors, platformDispatcher.onError catches async errors (returns true to prevent crash), ErrorWidget.builder shows user-friendly error UI instead of red error screen.
- * **Firestore Init (lines 46-67):** Skipped in demo mode. Loads .env variables, initializes Firestore with hardcoded FirestoreOptions (apiKey, appId, projectId).
- * **StorageService Init (lines 69-76):** Initializes Hive local database. In demo mode, auto-completes onboarding.
- * **ProviderScope (line 79):** Wraps entire app in Riverpod's ProviderScope for dependency injection.
- * **MindSyncApp (lines 85-106):** ConsumerWidget that watches GoRouter provider and inactivity service. Uses MaterialApp.router with system theme mode (light/dark auto-detection).

31-34. Core Infrastructure

Module	File(s)	Purpose
Config	app_config.dart	Environment config: backend URL, API keys, endpoints, breakpoints. demoMode flag toggles offline mode.
Routing	app_router.dart (235 lines)	GoRouter with 20 routes, StatefulShellRoute for bottom nav (5 tabs: Dashboard, Mood, Chat, Reports, Profile). Auth-based redirects.
Theme	app_theme.dart + app_colors.dart	Material 3 theme system with light/dark modes. Google Fonts (Inter). Custom card, button, input styles.
Network	dio_client.dart + auth_interceptor.dart	Dio HTTP client with 15s timeouts, auto-auth header injection from Hive, error mapping to typed exceptions.

35. Feature Modules - Clean Architecture Pattern

Every feature module follows a **3-layer Clean Architecture** pattern:

Layer	Directory	Contents	Responsibility
Data	data/	models/, datasources/, repositories/	API communication, JSON serialization, data source abstraction
Domain	domain/	entities/, repositories/	Business entities (pure Dart classes), repository interfaces
Presentation	presentation/	pages/, providers/, widgets/	UI screens, Riverpod state management, reusable widgets

36-43. Feature Module Summary

Feature	Pages	Key Functionality
Authentication	Splash, Onboarding, Login, Register, ForgotPassword	Firebase Auth (email/password), JWT token storage in Hive, auto-redirect based on auth state
Dashboard	DashboardPage, MainLayoutPage + 11 widget cards	Welcome header, wellness score card, burnout risk, mood summary, activity summary, recommendations, weather, motivational quotes, weekly charts

Feature	Pages	Key Functionality
Mood Tracking	MoodLogging, MoodJournal, MoodStatistics	Emoji mood selector, stress/energy sliders, sleep/water/exercise input, calendar view, statistics charts
AI Chat	ChatPage + bubble/drawer/prompts widgets	Real-time AI wellness coaching, conversation history, suggested prompts, developer-friendly analogies
Burnout	Data/domain models + providers	Displays ML burnout predictions, risk scores, SHAP-identified factors
Recommendations	RecommendationsPage + card/feedback widgets	Shows hybrid recommendations, completion tracking, save/feedback functionality
Reports	ReportsPage + 10 chart widgets	Time-filtered analytics, 8 chart types (mood/stress/sleep/hydration/exercise/burnout/goals/wellness), PDF export
Notifications	NotificationCenter + Settings pages	Notification list with read/delete, preferences (push, email, quiet hours, frequencies)
Profile	ProfilePage	User profile display/edit, photo upload, account management
Settings	Settings, Privacy, Security, DataManagement, HelpSupport, About	App preferences, privacy controls, security settings, data export/delete, help/FAQ, app info

PART IV: DATABASE & SECURITY

44. Firestore Database Schema & Security Rules (91 lines)

Database: Cloud Firestore (NoSQL document database by Google Firebase)

Collection	Document Key	Purpose	Security Rule
users	{userId}	User profiles and account data	Only owner can read/write (isOwner)
mood_logs	{logId}	Daily wellness metric entries	Auth users can read; only creator can modify/delete
burnout_predictions	{predId}	ML prediction results	Auth users can read; only creator can modify/delete
recommendations	{recId}	AI-generated recommendations	Auth users can read; only creator can modify/delete
reports	{reportId}	Generated wellness reports	Auth users can read; only creator can modify/delete
privacy_settings	{userId}	User privacy preferences	Only owner can read/write
application_settings	{userId}	App configuration per user	Only owner can read/write
chat_sessions	{sessionId}	AI chat conversation sessions	Only owner can CRUD
chat_messages	{msgId}	Individual chat messages	Only owner can CRUD
notifications	{notifId}	Push notification records	Auth users can read; only creator can modify/delete
notification_preferences	{userId}	Notification settings per user	Only owner can read/write
analytics_cache	{cacheId}	Cached analytics computations	Auth users can read/create; only creator can modify

Helper Functions: isAuthenticated() checks request.auth != null. isOwner(userId) verifies request.auth.uid matches the userId parameter.

PART V: DEPLOYMENT & DEVOPS

46. Docker Containerization

- * **Base Image:** python:3.12-slim (minimal Debian-based Python image)
- * **Build Stage:** Installs build-essential for native extensions, copies requirements.txt, pip installs dependencies
- * **Runtime:** Sets ENV=production, exposes port 8000, runs uvicorn app.main:app
- * **Health Check:** Built-in HEALTHCHECK every 30s using Python urllib (avoids curl dependency)
- * **docker-compose.yml:** Single-service setup for local development with volume mounts and env vars

47. Render Cloud Deployment

- * **render.yaml:** Infrastructure-as-Code definition: web service, Docker runtime, free plan, /health check endpoint
- * **Environment Variables:** ENV=production, ALLOWED_ORIGINS=*, RATE_LIMIT_PER_MINUTE=60, SECURE_HEADERS_ENABLED=true
- * **Secrets:** GEMINI_API_KEY and FIREBASE_CREDENTIALS_JSON stored as Render secrets (not in code)
- * **Live URL:** <https://ai-mental-wellness-companion-for-it.onrender.com>

PART VI: TESTING

48. Backend Test Suite (7 Test Files, 25+ Test Cases)

File	Tests	What It Validates
test_api.py	3 tests	Health endpoint returns 200+healthy; predict endpoint rejects unauthorized; predict endpoint returns correct schema with mock token
test_security.py	3 tests	Prompt injection patterns are detected and blocked; medical disclaimer is appended; unauthorized endpoints return 401
test_mlops.py	4 tests	ModelManager initialises and reports health; CacheService TTL expiry works; AI orchestrator resolves; detailed health endpoint returns all components
test_recommendations.py	2 tests	Recommendation generation returns rules-based items; feedback submission succeeds
test_analytics.py	7 tests	Summary computes correct wellness scores; trends return filtered data; historical stats compare weeks/months; report generation works; PDF/CSV export produces correct content-type
test_notifications.py	5 tests	Unauthorized access blocked; default notifications seeded; mark-as-read works; delete works; trigger-evaluation generates alerts from high-stress metrics
test_production.py	2 tests	Security headers present (X-Frame-Options=DENY); validation errors return standardized response format with requestId and timestamp

PART VIII: COMPLETE USER GUIDE - ALL FEATURES

52. Feature-by-Feature User Guide

1. Splash & Onboarding

When you first open the app, you see a splash screen with the MindSync AI logo. If it's your first time, you're taken through an onboarding carousel explaining the app's key features: AI burnout prediction, mood tracking, wellness recommendations, and AI chat coaching.

2. Registration & Login

Create an account with email/password via Firebase Authentication. The app stores your Firebase ID Token in Hive local storage for automatic re-login. Forgot Password sends a password reset email via Firebase.

3. Dashboard (Home Screen)

The main hub showing 11 widget cards: Welcome Header (greeting with name), Wellness Score Card (0-100 daily score), Burnout Risk Card (Low/Medium/High from ML model), Mood Summary Card (today's mood emoji), Activity Summary (steps, exercise), Recommendations Card (top AI suggestions), Weather Widget (local weather), Motivational Quote, Weekly Wellness Charts, Recent Mood Entries, and Quick Actions Grid.

4. Mood Logging

Tap the + button or 'Log Mood' quick action. Select a mood emoji from a 5-option grid. Adjust Stress Level (1-10 slider) and Energy Level (1-10 slider). Enter Sleep Hours, Water Intake (glasses), Exercise Minutes. Add optional journal notes. Submit saves to Firestore and triggers backend burnout prediction.

5. Mood Journal

View all past mood entries in a scrollable list. Each entry shows date, mood emoji, scores, and notes. Tap to expand details. Calendar view shows mood emojis on each logged day. Filter by date range.

6. Mood Statistics

Visualise your mood trends with interactive charts: mood score trend line, stress level bar chart, sleep hours area chart, hydration tracking, exercise consistency. Filter by 7 days, 30 days, or 90 days.

7. AI Burnout Prediction

After logging mood, the app automatically calls the /predict/burnout API. The Random Forest model analyses your 9 metrics and returns: Risk Level (Low/Medium/High with colour coding), Confidence Score (%), Risk Score (0-100), Top 3 Contributing Factors (from SHAP analysis, e.g., 'Low sleep duration', 'Elevated stress index'), and personalised recommendations.

8. AI Recommendations

The Recommendations page shows a list of cards from three sources: Rules Engine (deterministic, based on thresholds), Machine Learning (based on burnout risk level), and Groq AI (creative, personalised suggestions from LLaMA 3.3-70B). Each card shows title, description, reason, priority, estimated time, difficulty, and expected benefit. You can mark recommendations as completed, save them, or provide feedback.

9. AI Chat Coach

A conversational AI wellness coach. Type messages and receive supportive responses using developer-friendly analogies (e.g., 'Think of stress like memory leaks -- you need regular garbage collection'). The chat uses Groq LLM with system prompt: 'You are MindSync, a compassionate AI wellness coach designed for software engineers.' Conversation history is stored in Firestore. Suggested prompts help you get started.

10. Reports & Analytics

A comprehensive analytics dashboard with 8 chart types: Mood Trend Chart, Stress Trend Chart, Sleep Quality Chart, Hydration Chart, Exercise Chart, Burnout Risk Chart, Goals Dashboard, and Wellness Score Analytics. Time filter selector (Today, 7 days, 30 days, 90 days, Custom). Generate Report button creates a full wellness report that can be exported as PDF or CSV.

11. Notifications

The Notification Center shows all alerts: burnout warnings, sleep reminders, hydration reminders, exercise prompts, mood check-ins, and general wellness tips. Swipe to dismiss or tap to mark as read. Notification Settings let you configure: push/email toggles, daily reminders, weekly summaries, AI suggestions, motivation messages, goal reminders, sound/vibration, wake-up/sleep times, water frequency, quiet hours.

12. Profile

View and edit your profile: display name, email, profile photo (uploaded to Firebase Storage). Account management options.

13. Settings

Application Settings: theme, language, demo mode toggle. Privacy Settings: data sharing, analytics opt-in/out, visibility controls. Security Settings: password change, session management. Data Management: export data (JSON/CSV), delete account and all data. Help & Support: FAQ, contact, feedback. About: version info, licenses, credits.

PART IX: VIVA PREPARATION - 50 Q&A

Q1: What is MindSync AI?

MindSync AI is a production-grade, AI-driven mental wellness companion mobile application designed specifically for software engineers and IT professionals. It uses Machine Learning to predict burnout risk, Generative AI to provide personalised wellness recommendations, and a Rules Engine for instant threshold-based alerts.

Q2: What problem does it solve?

IT professionals face high rates of occupational burnout (83% of developers report burnout). Traditional wellness apps are generic. MindSync specifically targets developer stress patterns: long screen hours, sedentary coding sessions, deadline pressure, and irregular sleep from late-night debugging.

Q3: What is the technology stack?

Backend: Python FastAPI, scikit-learn, SHAP, Groq LLM API. Frontend: Flutter (Dart), Riverpod state management, GoRouter. Database: Firebase (Firestore + Auth). Deployment: Docker, Render cloud. ML Model: Random Forest trained on synthetic data.

Q4: What ML algorithm is used and why?

Random Forest Classifier. It was selected over Logistic Regression, Decision Tree, and Gradient Boosting because: (1) it handles non-linear feature relationships, (2) it is resistant to overfitting due to bagging, (3) it provides probability estimates for confidence scoring, (4) it is compatible with SHAP TreeExplainer for fast explainability.

Q5: How is the training data generated?

We use synthetic data generation with NumPy. 10,000 records are created with 9 features using uniform and randint distributions mimicking realistic wellness ranges. A domain-expert-weighted formula computes risk scores, with Gaussian noise added to prevent deterministic class boundaries.

Q6: What are the 9 features used for prediction?

sleep_hours, working_hours, mood_score, stress_level, energy_level, water_intake, daily_steps, exercise_minutes, and consecutive_working_days. These represent the key behavioral and wellness metrics of IT professionals.

Q7: What is RobustScaler and why use it?

RobustScaler normalises features using median and IQR (interquartile range) instead of mean and standard deviation. It is resistant to outliers -- important because features like working_hours can have extreme values (16-hour days) that would skew standard scaling.

Q8: What is GridSearchCV?

GridSearchCV is an exhaustive hyperparameter tuning method that tests all combinations of specified parameter values. We test 18 combinations (3 n_estimators x 3 max_depth x 2 min_samples_split) with 3-fold cross-validation, scoring by macro-averaged F1-score.

Q9: What is SHAP and how is it used?

SHAP (SHapley Additive exPlanations) is a game-theoretic approach to explain ML predictions. We use TreeExplainer (optimised for tree-based models) to compute each feature's contribution to a specific prediction. The top 3 positive contributors are shown to users as 'Important Factors' (e.g., 'Low sleep duration', 'Elevated stress index').

Q10: What is the difference between local and global explainability?

Local explainability explains a single prediction (why THIS user got High risk). Global explainability explains overall model behaviour (which features are generally most important). SHAP provides both: per-prediction values (local) and summary plots (global).

Q11: How does the Hybrid Recommendation Engine work?

It combines 3 sources: (1) Rules Engine: deterministic if-then rules based on metric thresholds, (2) ML-Driven: recommendations based on burnout risk level from the Random Forest model, (3) Groq LLM: creative, context-aware suggestions generated by LLaMA 3.3-70B. If the LLM fails, a hardcoded fallback is used.

Q12: What LLM is used and why Groq?

We use Meta's LLaMA 3.3-70B-Versatile model via the Groq API. Groq provides ultra-fast LLM inference with an OpenAI-compatible API, making it easy to integrate. The 70B parameter model provides high-quality, nuanced wellness advice.

Q13: What is prompt engineering in MindSync?

We design structured prompts with: (1) Persona assignment ('You are an expert IT wellness coach'), (2) Output format constraints (JSON schema), (3) Domain anchoring (developer-specific language), (4) Guardrails ('Do not provide clinical diagnostics'). Templates are centralised in PromptLibrary.

Q14: How is AI security implemented?

Three defence layers: (1) HTML tag stripping prevents XSS, (2) 7 regex patterns block prompt injection attempts (e.g., 'ignore all prior instructions'), (3) Medical disclaimer is auto-appended to all AI responses. Blocked requests return HTTP 400.

Q15: How does authentication work?

Firebase Authentication handles user registration and login (email/password). Firebase issues JWT ID tokens. The backend verifies tokens using `firebase_admin.auth.verify_id_token()`. A manual JWT decode fallback handles cases where Firebase Admin lacks full credentials on cloud deployment.

Q16: What middleware is used?

Four middleware layers: (1) `CORSMiddleware` for cross-origin requests, (2) `RequestLoggingMiddleware` for audit trails, (3) `RateLimitingMiddleware` (60 req/min/IP sliding window), (4) `SecurityHeadersMiddleware` (6 OWASP headers).

Q17: What is the Singleton pattern and where is it used?

The Singleton pattern ensures only one instance of a class exists. Used in `BurnoutPipeline`, `ModelManager`, `BackgroundScheduler`, and `CacheService`. Prevents redundant model loading (the 4.8MB model is loaded once at startup).

Q18: How does the caching system work?

`CacheService` is an in-memory TTL cache. Keys map to (value, expiry_timestamp) tuples. Expired entries are auto-deleted on access. Daily summaries cache for 1 hour, weekly reports for 12 hours. Nightly cleanup job clears all cache.

Q19: What is the deployment architecture?

The backend is containerised with Docker and deployed to Render cloud platform. `render.yaml` defines the service config: Docker runtime, free plan, /health check endpoint, and environment variables. Firebase handles the database and auth layer.

Q20: What testing framework is used?

pytest for Python backend. 7 test files with 25+ test cases covering: API endpoints, security, ML model operations, recommendations, analytics, notifications, and production config. FastAPI's `TestClient` is used for HTTP request simulation.

Q21: What is the wellness score formula?

Daily Wellness Score = $(30\% \times \text{Mood}/10 \times 100) + (20\% \times (10 - \text{Stress})/9 \times 100) + (20\% \times \min(\text{Sleep}, 8)/8 \times 100) + (15\% \times \min(\text{Water}, 8)/8 \times 100) + (15\% \times \min(\text{Exercise}, 30)/30 \times 100)$. Range: 0-100.

Q22: How does the risk score differ from the risk class?

Risk Class is discrete: Low/Medium/High (from `model.predict()`). Risk Score is continuous 0-100 calculated from probabilities: $\text{riskScore} = P(\text{Medium}) \times 50 + P(\text{High}) \times 100$. The continuous score provides more granular assessment.

Q23: What is Clean Architecture in the Flutter app?

Each feature module has 3 layers: Data (API communication, JSON models), Domain (pure business entities, repository interfaces), Presentation (UI pages, Riverpod providers, widgets). Dependencies point inward: Presentation -> Domain <- Data.

Q24: What state management is used?

`flutter_riverpod` for reactive state management and dependency injection. Providers watch auth state, API responses, and local data. `ConsumerWidget` and `ConsumerStatefulWidget` react to state changes.

Q25: How does offline support work?

Hive (fast NoSQL key-value store) caches data locally. Mood logs, predictions, and recommendations are stored in Hive boxes. When connectivity is restored, data syncs to Firestore. The demo mode works entirely offline.

Q26: What is the model artifact (.pkl file)?

`burnout_model.pkl` (~4.8MB) contains: the trained `RandomForestClassifier`, the fitted `RobustScaler`, the list of 9 feature names, and metadata (version, training date, algorithm name). Created by `joblib.dump()` during training.

Q27: Why is SHA-256 checksum computed for the model?

`ModelManager` computes SHA-256 hash of the `.pkl` file before loading to verify integrity. If the model file is corrupted during deployment or tampered with, the checksum would change, alerting the system.

Q28: What happens if the ML model fails to load?

`ModelManager` activates fallback heuristics: `model=None`, a simplified rule-based system provides basic predictions with estimated 80% accuracy. The app never crashes -- it degrades gracefully.

Q29: How is the Firestore security structured?

Row-Level Security: each document has a `userId` field. Security rules enforce that users can only read/write their own data. Helper functions `isAuthenticated()` and `isOwner()` are reused across all 12 collections.

Q30: What charts does the reports page show?

8 charts: (1) Mood Trend Line, (2) Stress Trend Line, (3) Sleep Quality Area, (4) Hydration Bar, (5) Exercise Bar, (6) Burnout Risk Line, (7) Goals Dashboard Pie, (8) Wellness Score Analytics. All powered by `fl_chart` package.

Q31: What is exponential backoff?

A retry strategy where wait time doubles after each failure: $2^0=1\text{s}$, $2^1=2\text{s}$, $2^2=4\text{s}$. Used in `AIOrchestrator` when Groq API calls fail. Prevents overwhelming a struggling service.

Q32: How does the notification evaluation work?

The trigger-evaluation endpoint accepts current wellness metrics, passes them to RuleBasedNotificationEngine.evaluate_rules(), which checks 6 threshold conditions and generates notification payloads with type, priority, and contextual messages.

Q33: What is the difference between training and inference?

Training: learning patterns from data (done once, offline, by train.py). Inference: using the trained model to make predictions on new data (done at runtime, per API request, by pipeline.py). The trained model is serialised and loaded for inference.

Q34: Why use synthetic data instead of real data?

Real mental health data is: (1) protected under HIPAA/GDPR regulations, (2) extremely difficult to collect at scale, (3) requires ethical review board approval. Synthetic data avoids all these issues while maintaining realistic statistical distributions.

Q35: What is cross-validation?

A technique to evaluate model performance by splitting training data into k folds, training on k-1 folds, and validating on the held-out fold. Rotated k times. We use 3-fold CV in GridSearchCV. It prevents overfitting to a single train/test split.

Q36: What is F1-score and why use macro?

F1-score is the harmonic mean of precision and recall. Macro averaging computes F1 for each class independently and then averages them equally. This is important for imbalanced classes where accuracy alone could be misleading.

Q37: How does the chat feature use prompt engineering?

The chat_coaching template sets the persona: 'You are MindSync, a compassionate AI wellness coach designed for software engineers.' It instructs the LLM to use developer analogies (compile times, debugging, memory leaks) and prohibits clinical diagnostics.

Q38: What is the role of Pydantic in the backend?

Pydantic provides automatic request validation. Each endpoint has a BaseModel schema with Field() constraints (ge=0, le=24 for sleepHours). Invalid requests are automatically rejected with structured 422 error responses before reaching business logic.

Q39: How does the app handle network errors?

DioClient maps DioException types to custom exceptions: timeout -> NetworkException, HTTP errors -> ServerException with status code and message. The UI shows user-friendly error messages instead of technical details.

Q40: What is GoRouter and how is auth routing handled?

GoRouter is a declarative routing package. The router watches authStateProvider. The redirect callback checks: if authenticated user tries to access login -> redirect to dashboard; if unauthenticated user tries to access protected page -> redirect to login.

Q41: How is the wellness score different from burnout risk?

Wellness Score (0-100) is a formula-based calculation from raw metrics. Burnout Risk (Low/Medium/High) is an ML prediction from the Random Forest model. They often correlate but use different methodologies: formula vs learned patterns.

Q42: What design patterns are used?

Singleton (BurnoutPipeline, ModelManager), Repository Pattern (data layer abstraction), Provider Pattern (Riverpod DI), Observer Pattern (Riverpod state watching), Strategy Pattern (3 recommendation sources), Decorator Pattern (middleware chain), Template Method (prompt templates).

Q43: How does PDF export work?

ReportLab library generates PDF documents with custom ParagraphStyles, Tables with TableStyle formatting, Spacers, and styled content. The report includes a wellness score box, AI insights section, and a metrics telemetry table. Returned as StreamingResponse with content-disposition attachment header.

Q44: What is the model training accuracy?

The tuned Random Forest typically achieves ~93% accuracy and ~0.93 macro F1-score on the test set. This is printed during training. The classification report shows per-class precision, recall, and F1.

Q45: How does water intake conversion work?

Users may log water in millilitres (e.g., 1500ml) or glasses (e.g., 6). If the value exceeds 30, it's treated as millilitres and divided by 250 to convert to glasses. This normalisation happens in predict.py, recommendations.py, and analytics.py.

Q46: What is the role of the BackgroundScheduler?

Two async background loops: (1) Nightly cleanup clears CacheService every 24 hours, (2) Recommendation refresh pre-calculates recommendations every 12 hours. Uses asyncio.create_task() for non-blocking execution alongside the API server.

Q47: How does demo mode work?

When AppConfig.demoMode = true, the app skips Firebase initialization, auto-completes onboarding, and uses local mock data. All features work with hardcoded demo data. This is useful for presentations and testing without a backend.

Q48: What security headers are added and why?

X-Frame-Options: DENY (prevents clickjacking), X-Content-Type-Options: nosniff (prevents MIME sniffing), X-XSS-Protection (browser XSS filter), HSTS (forces HTTPS), CSP (restricts resource sources), Referrer-Policy: no-referrer (prevents information leakage).

Q49: How is the project different from existing wellness apps?

Three key differentiators: (1) IT-specific: designed for developer stress patterns, not generic wellness, (2) Explainable AI: SHAP tells users WHY they are at risk, not just that they are, (3) Hybrid AI: combines ML prediction + LLM creativity + deterministic rules for reliability.

Q50: What future improvements could be made?

Real clinical data collection with IRB approval, federated learning for privacy-preserving model updates, wearable device integration (smartwatch heart rate/HRV), multi-language LLM support, team-level burnout analytics for engineering managers, and a React web dashboard for desktop users.

This document covers 100% of the MindSync AI codebase, architecture, and functionality. Good luck with your viva and presentation!